UNIVERSIDAD DEL QUINDÍO

Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

MCU Uniquindio Project: Specification

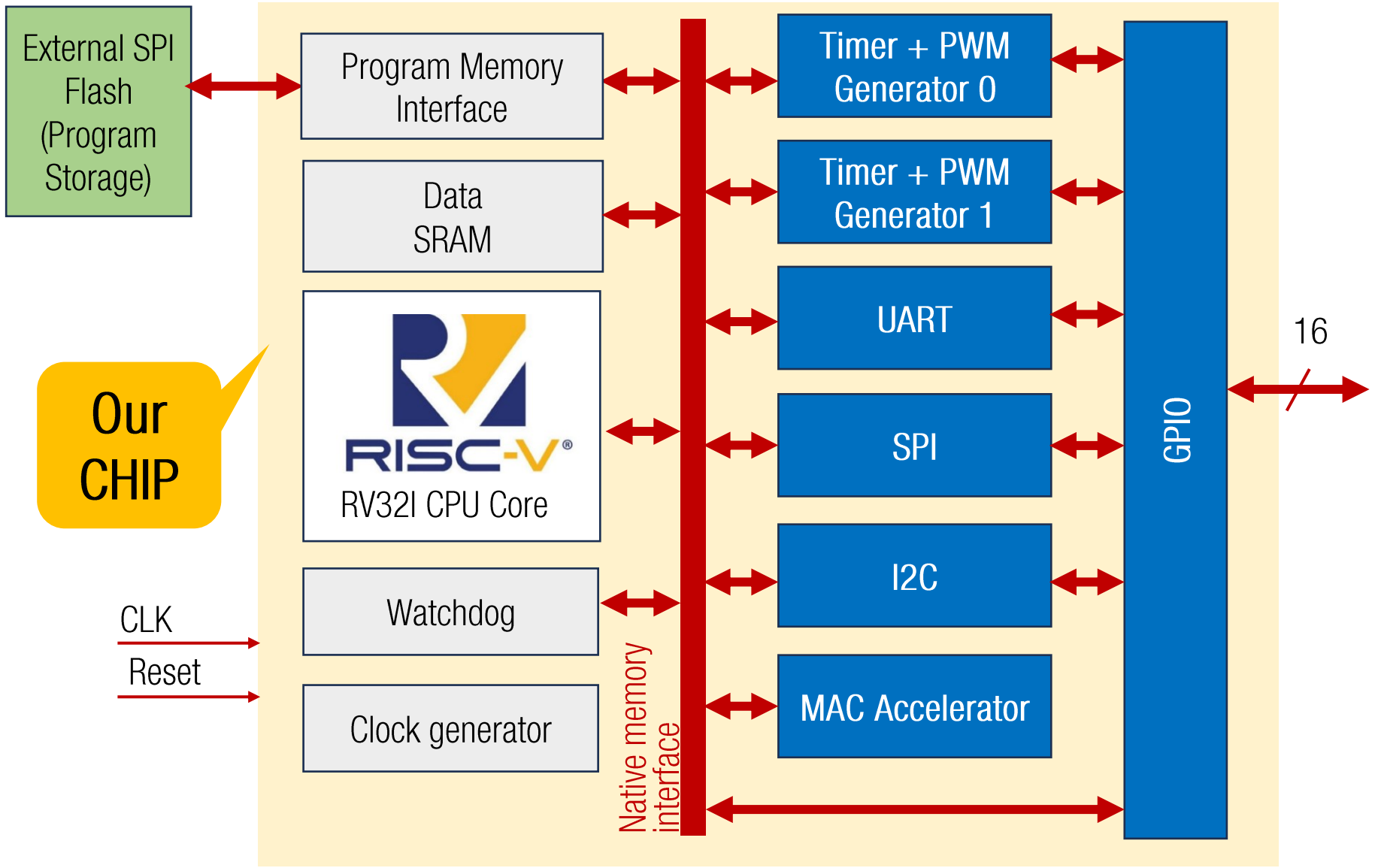
Programa de Ingeniería Electrónica

Alexander Vera y Jorge Iván Marín

UNIQUINDÍO
en conexión territorial

www.uniquindio.edu.co

General Diagram



- MCU based on RV32I specification
- Verilog for RV32I from the picoRV32 project: https://github.com/YosysHQ/picorv32
  (No AXI, no Wishbone, no co-processor PCPI, IRQ support: yes)
- Blue modules will be designed in teams
  - Please check PicoRV32 Native Memory Interface

Data memory map

Addr	
0x0000 0000	I/O Space
0x1000 0000	SRAM

Address	Register Name	Module
0x00	Reserved	Watchdog
0x04	Reserved	
0x08	WATCHDOG	
0x0C	WATCHDOG_CTRL	
0x10	GPIO_DIR	GPIO Interface
0x14	GPIO_DATA_OUT	
0x18	GPIO_DATA_IN	
0x1C	GPIO_CTRL	
0x20	TIMER_CNT0	TIMER 0
0x24	TIMER_TOP0	
0x28	PWM_CNTA0	
0x2C	PWM_CNTB0	
0x30	TIMER_CTRL0	

Address	Register Name	Module
0x40	TIMER_CNT1	TIMER 1
0x44	TIMER_TOP1	
0x48	PWM_CNTA1	
0x4C	PWM_CNTB1	
0x50	TIMER_CTRL1	
0x54	Reserved	-
0x58	Reserved	-
0x5C	Reserved	-
0x60	UART_BITRATE	UART
0x64	UART_DATA_OUT	
0x68	UART_DATA_IN	
0x6C	UART_CTRL	

Address	Register Name	Module
0x70	I2C_BITRATE	I2C
0x74	I2C_DATA_OUT	
0x78	I2C_DATA_IN	
0x7C	I2C_CTRL	
0x80	SPI_BITRATE	SPI
0x84	SPI_DATA_OUT	
0x88	SPI_DATA_IN	
0x8C	SPI_CTRL	
0x90	MAC_INA	MAC
0x94	MAC_INB	
0x98	MAC_ACCL	
0x9C	MAC_ACCH	
0xA0	MAC_OUT	
0xA4	MAC_CTRL	

Program memory map

Addr	
0x30000000	Reset Vector
0x30000010	Interrupt Handler
0x30000200	Program

IRQ_n	ISR Vector
0	PicoRV32 – Timer
1	PicoRV32 – EBREAK/ECALL or Illegal Instruction
2	PicoRV32 – BUS Error (Unaligned Memory Access)
3	INT0
4	INT1
5	PINCHANGE
6	TIMER0
7	TIMER1
8	I2C
9	SPI
10	RXD
11	TXD
12	MAC

GPIO-Interface Specification

General Purpose I/O

UNIQUINDÍO
en conexión territorial

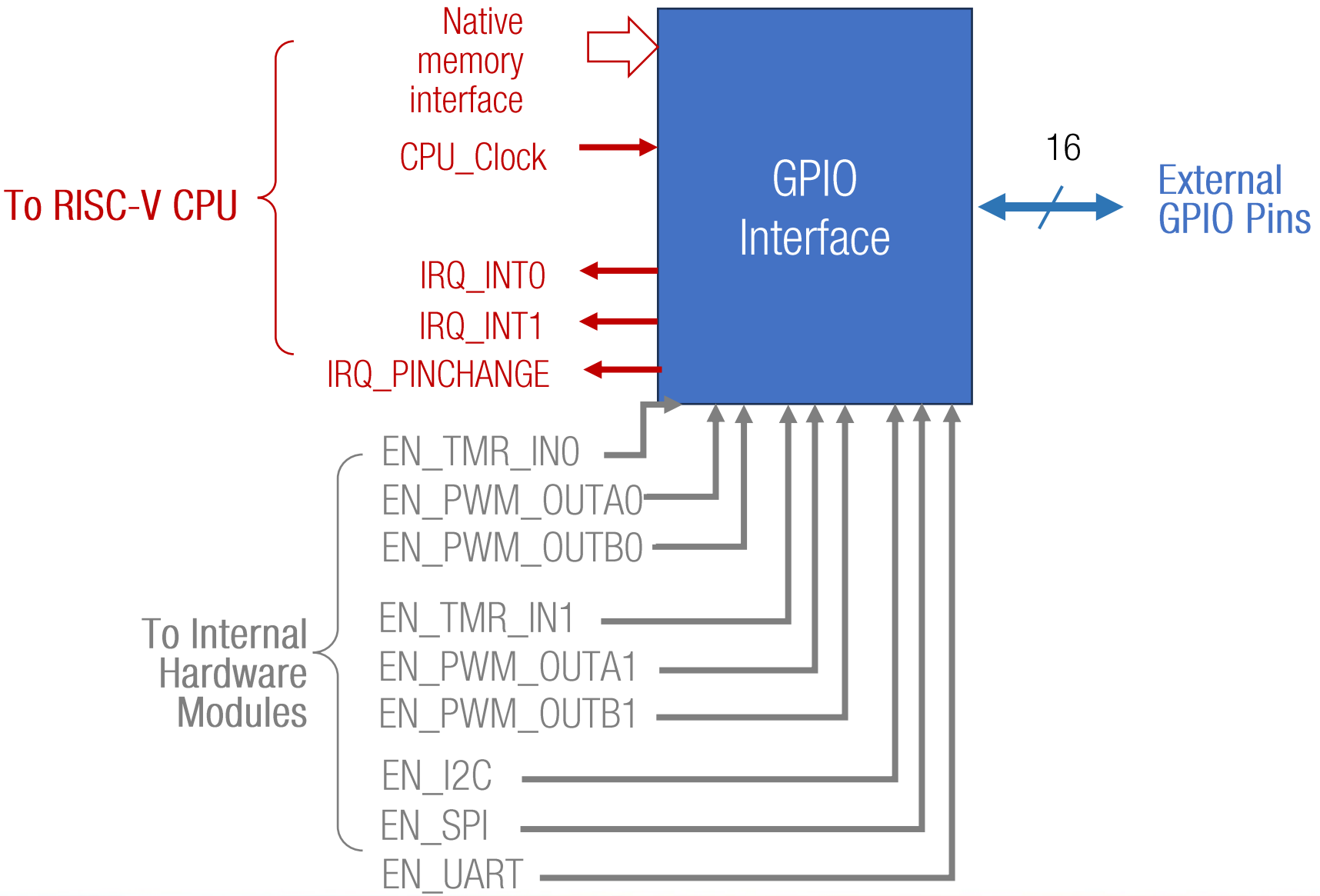
GPIO: General Purpose I/O

- 16 GPIO pins
- All GPIO can be individually configured as inputs or outputs
- Some GPIO pins share special functions depending on the hardware configuration
- Two external edge-triggered interrupts
- All GPIO pins can generate pin-change interrupt

15	14	13	12	11	10	9	8
INT1	INT0	TMR_IN1	PWM_OUTA1	PWM_OUTB1	TMR_IN0	PWM_OUTA0	PWM_OUTB0
Edge-Triggered Interrupts		Timer1			Timer0		

7	6	5	4	3	2	1	0
SPI_SCLK	SPI_MISO	SPI_MOSI	SPI_SS	I2C_SCL	I2C_SDA	UART_TXD	UART_RXD
SPI				I2C		UART	

GPIO: General Purpose I/O



GPIO Registers

Register name	Bit size	Description
GPIO_DIR	16	Each bit in this register set the direction of GPIO pin: • 0: input • 1: output These bits are ignored when a special module is enabled (See shared functions in the GPIO pins)
GPIO_DATA_OUT	16	All bits written in this register will be latched and output to the GPIO pins configured as outputs. These bits are ignored when a special module is enabled If this register is read, it retrieves the previous written value
GPIO_DATA_IN	16	Reads the GPIO pins configured as inputs. Bits in the GPIO pins configured as outputs will be read as 1
GPIO_CTRL	32	GPIO interrupt mask

GPIO Registers

GPIO_CTRL

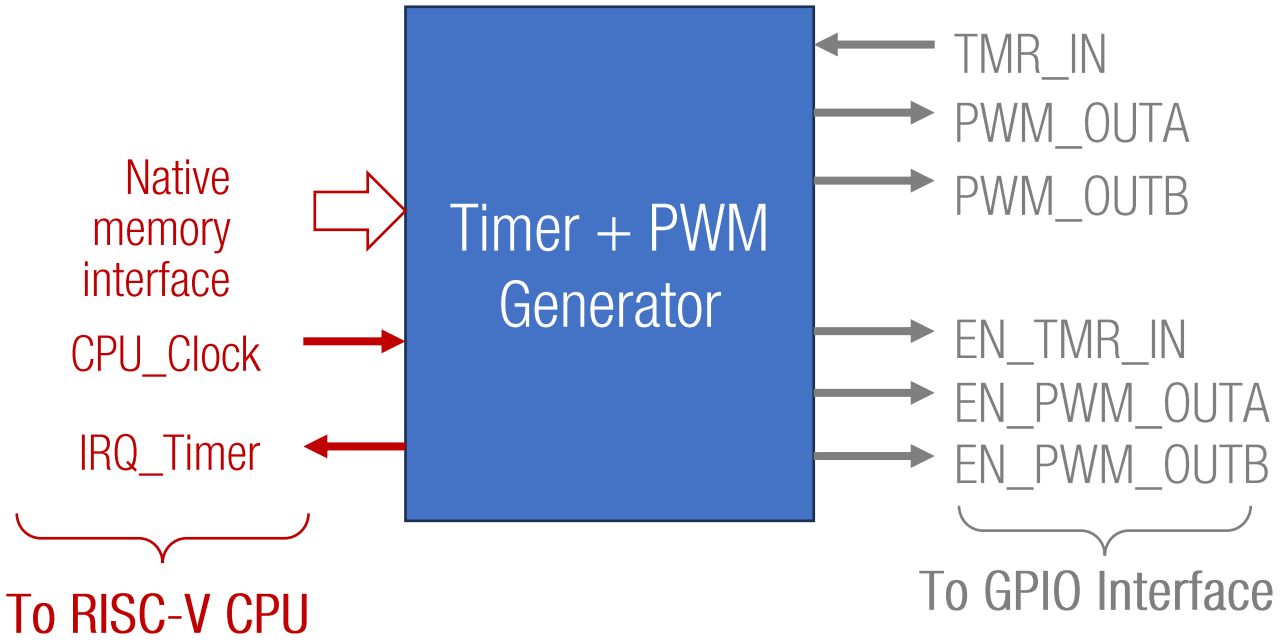
31	30	29	28	27-16	15-0
INT0_MSK		INT1_MSK		-	PINCHANGE_MSK

- INTx_MSK: External edge-triggered interrupt mask
  - 00: Disabled
  - 01: Rising
  - 10: Falling
  - 11: Change
- PINCHANGE_MSK: Pin-change interrupt mask
  - Individual bits state the GPIO input where pin- change interrupt can be triggered:
    - 0: Disabled
    - 1: Enabled

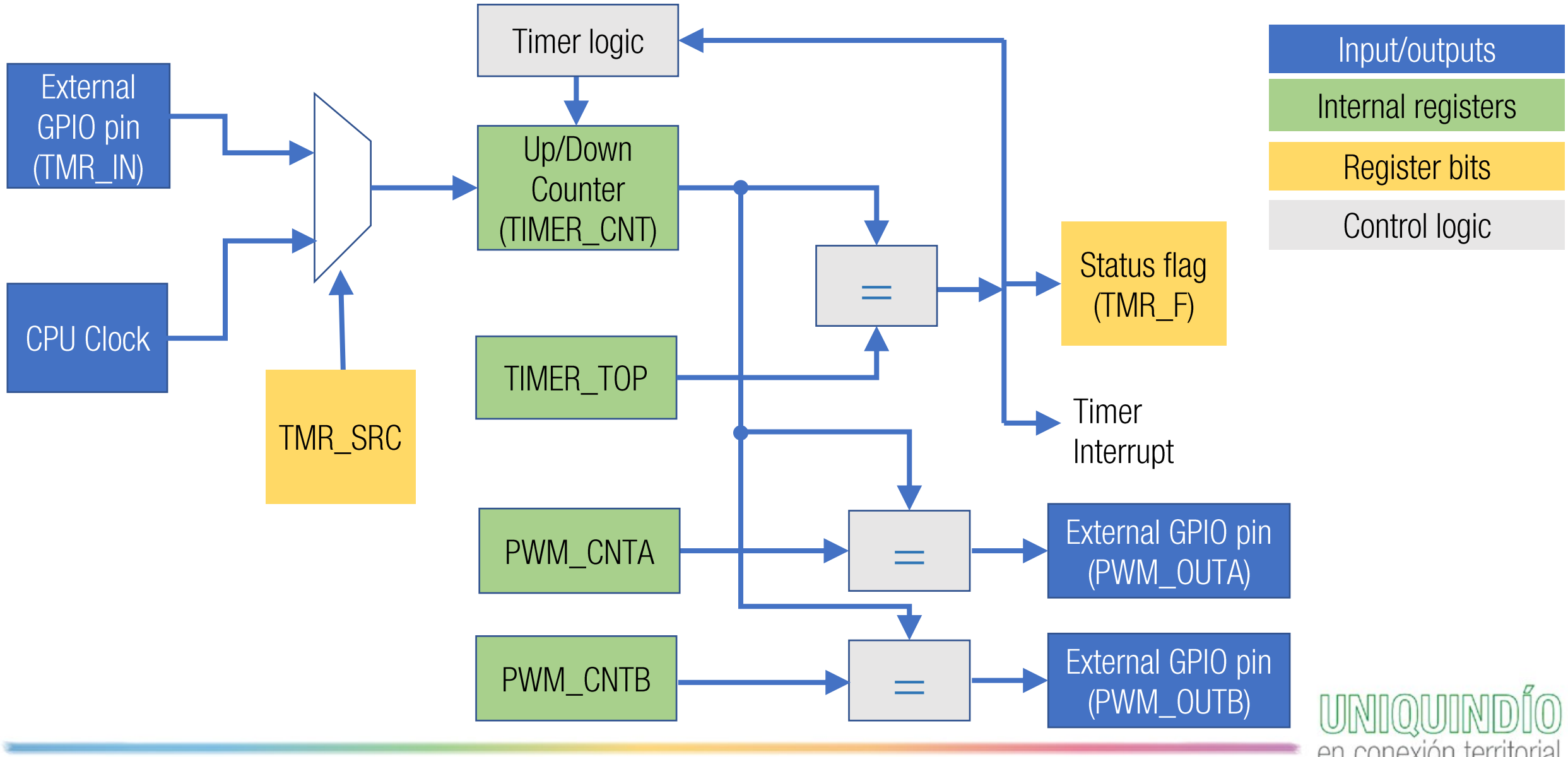
Timer + PWM Generator Specification

Timer + PWM Generator

- 32-bit Timer (TIMER_CNT)
- Input sources:
  - None (timer disabled)
  - Internal CPU clock
  - External GPIO pin (TMR_IN)
- When TIMER_CNT == TIMER_TOP:
  - Status flag=1 or
  - Timer interrupt
- Two PWM modes for timer output:
  - Fast mode: counting up
  - Phase correct mode: counting up/down
- Two PWM outputs:
  - PWM_OUTA toggles when TIMER_CNT==PWM_CNTA
  - PWM_OUTB toggles when TIMER_CNT==PWM_CNTB

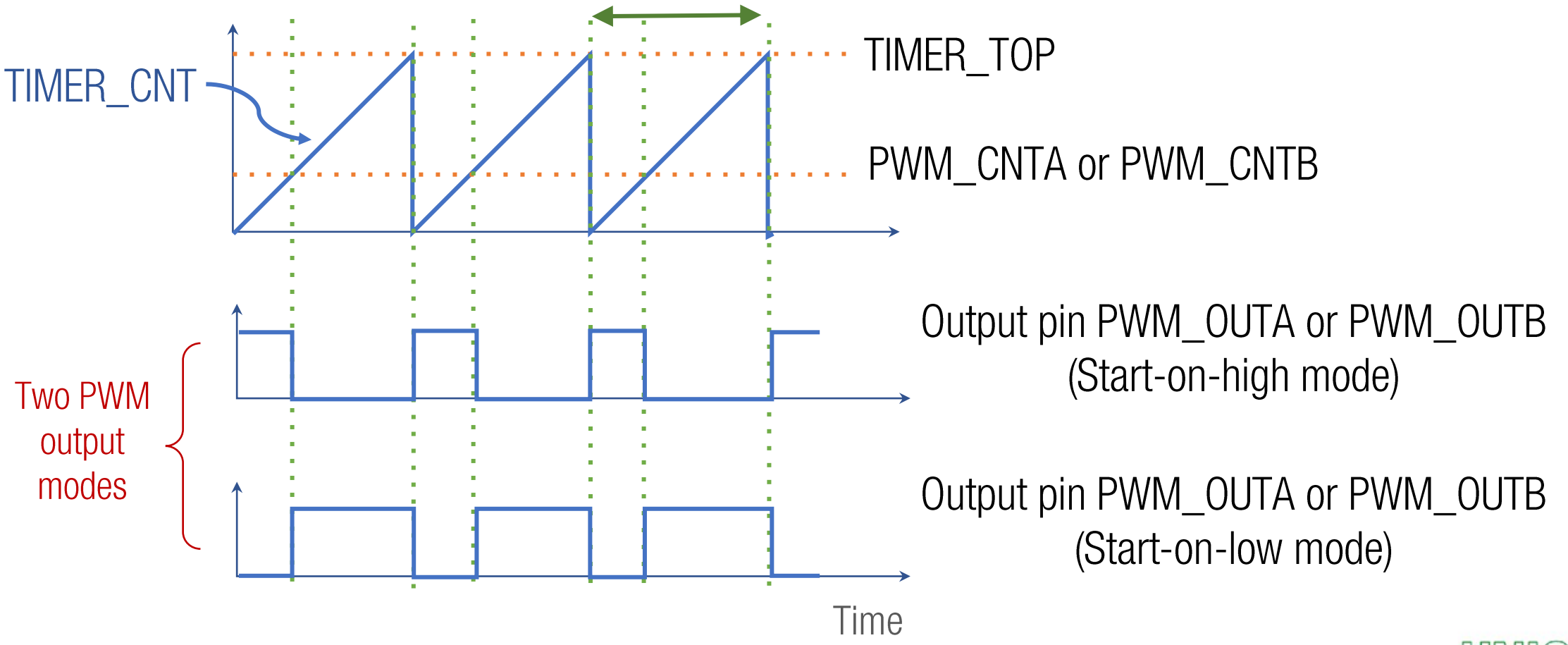


Timer + PWM Generator



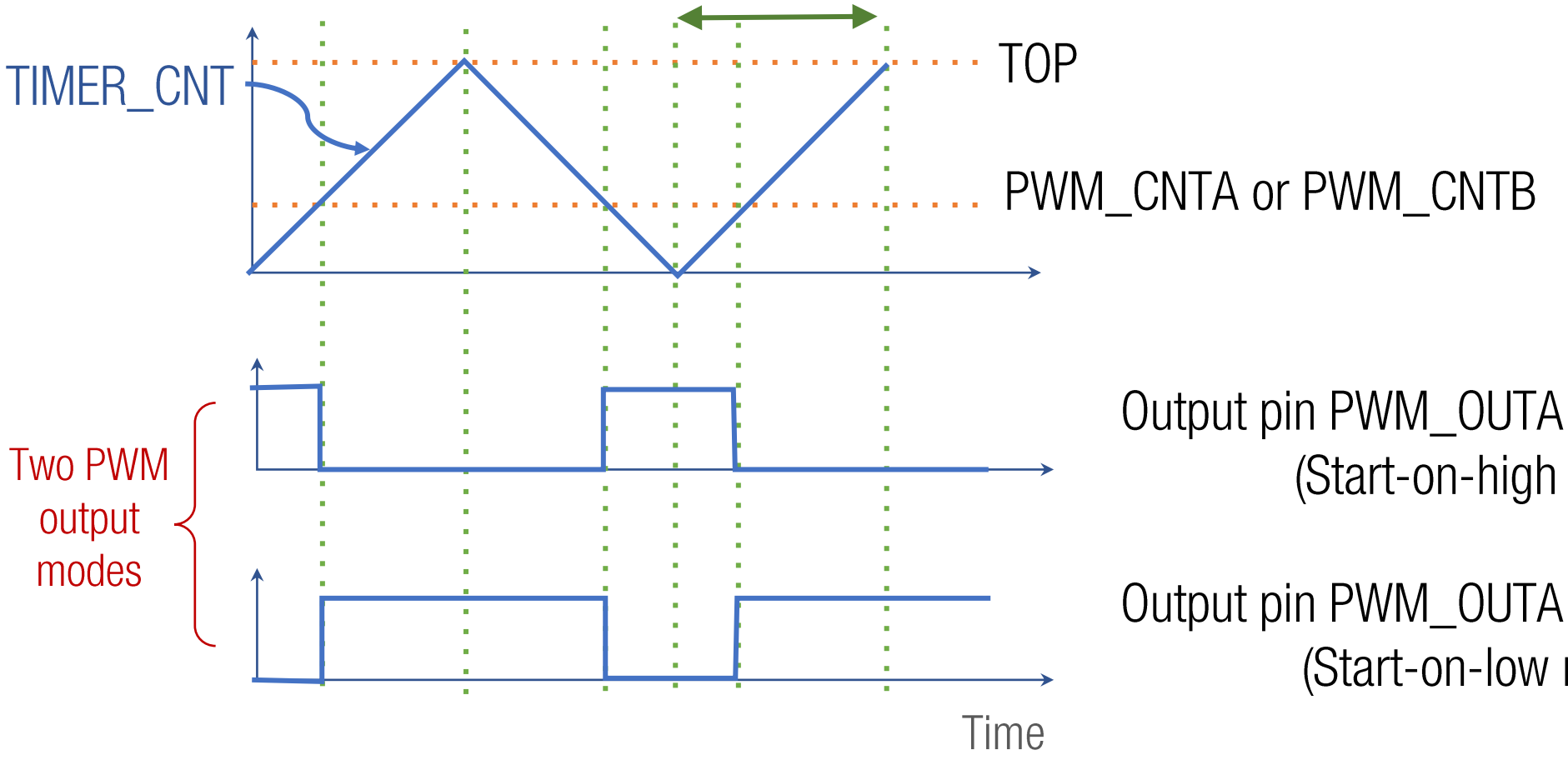
Fast PWM Mode

Period depends on TIMER_TOP value
(Counter is cleared when TIMER_CNT==TIMER_TOP)



Phase Correct PWM

Period depends on TIMER_TOP value
(Counter direction switches from up to down when TIMER_CNT==TIMER_TOP)



Output pin PWM_OUTA or PWM_OUTB
(Start-on-high mode)

Output pin PWM_OUTA or PWM_OUTB
(Start-on-low mode)

Timer Registers

Register name	Bit size	Description
TIMER_CNT	32	Timer counter
TIMER_TOP	32	When TIMER_CNT==TIMER_TOP: • In Normal mode: the timer status flag is set, and a timer interrupt is posted • In FastPWM mode: TIMER_CNT is cleared, and PWM_CNTA and PWM_CNTB outputs toggle • In PhaseCorrectPWM mode: Counting direction of TIMER_CNT changes.
PWM_CNTA	32	When TIMER_CNT==PWM_CNTA the timer output A toggles depending on the PWM mode
PWM_CNTB	32	When TIMER_CNT==PWM_CNTB the timer output B toggles depending on the PWM mode
TIMER_CTRL	10	Timer configuration and status flag

Timer Registers

TIMER_CTRL

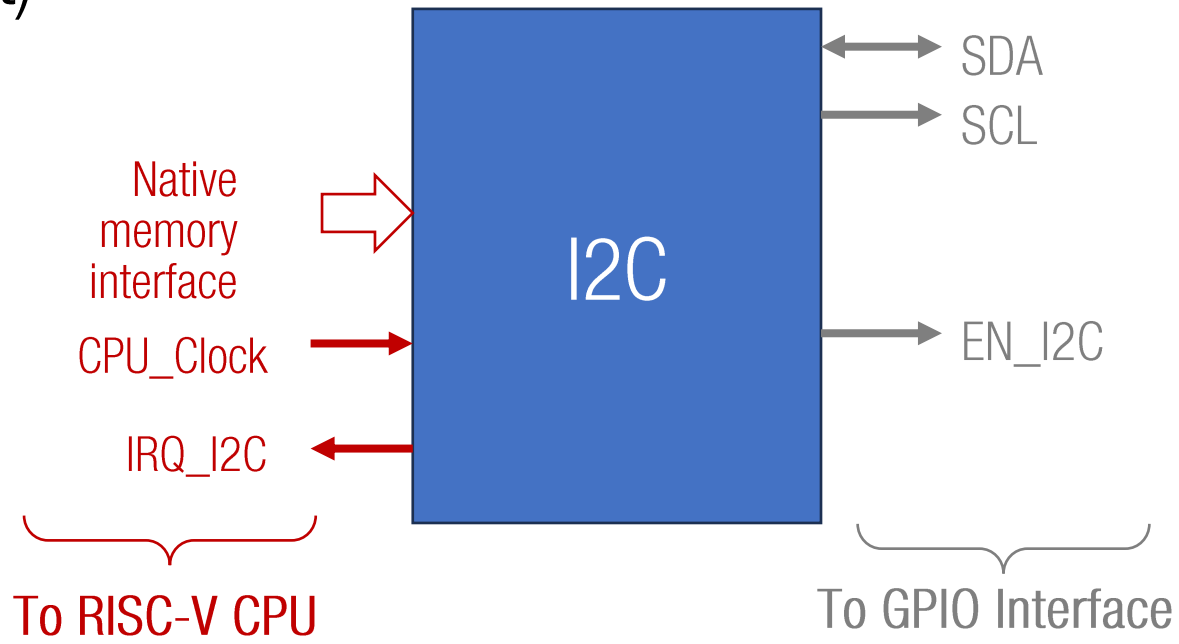
9	8	7	6	5	4	3	2	1	0
TMR_SRC		TMR_MODE		OUTA_MODE		OUTB_MODE		TMR_I_MSK	TMR_F

- TMR_SRC: Input source
  - 00: Disabled
  - 01: CPU Clock
  - 10: External input pin
  - 11: Reserved
- TMR_MODE: Timer mode
  - 00: Normal
  - 01: Fast PWM
  - 10: Phase Correct PWM
  - 11: Timer OFF
- OUTA_MODE / OUTB_MODE: Shape of the output signal A or B
  - 00: The output is disabled. The external pin associated to the PWM_OUTx pin behaves as a generic GPIO
  - 01: Start-on-high mode. The PWM_OUTx signal starts on high when TMR_CNT==0
  - 10: Start-on-low mode. The PWM_OUTx signal starts on low when TMR_CNT==0
  - 11: Reserved
  - Note: On the Start-on-high or Start-on-low modes, the state of the GPIO pin associated to the PWM_OUTx pin depends on the PWM signal generator.
- TMR_I_MSK: Timer interrupt mask.
  - When this bit is set, the timer interrupt is generated when TIMER_CNT==TIMER_TOP. When this bit is cleared, no interrupts are generated.
- TMR_F: Timer status flag.
  - If the timer interrupt is disabled, you must clear this flag when the TIMER_CNT register is set. The timer set this flag when TIMER_CNT==TIMER_TOP and it remains set until that you clear this flag.

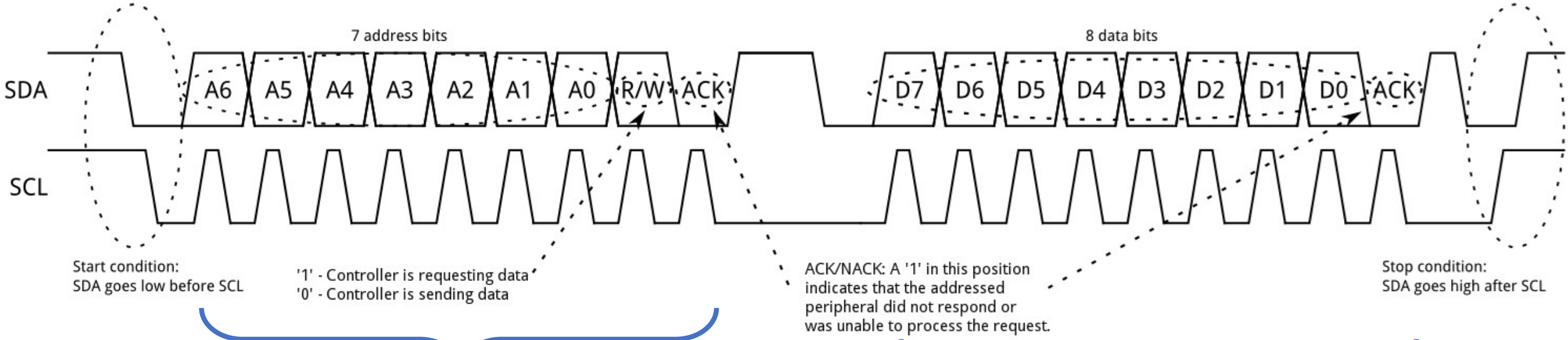
I2C Specification

I2C

- Work only as a master device
- Configurable I2C clock frequency
- Interrupts are generated when 8-bit data/address is transmitted (including ACK bit)



I2C (Timing diagram)



Slave address and Read/Write bit

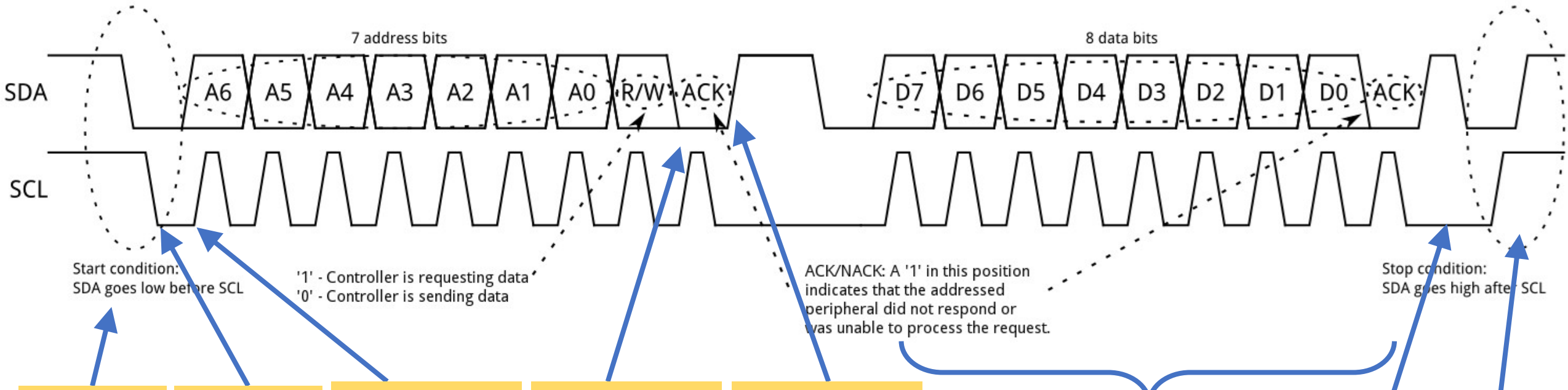
Multiple bytes (data frame) can be exchanged until the stop condition is reached.

Note that sending the slave address follows the same protocol of a data byte

Important: When reading data, master must set high SDA line

Based on: https://learn.sparkfun.com/tutorials/i2c/all

I2C (Registers & bit protocol during writing)



1. User set I2C_START flag

2. I2C module clears I2C_START flag when done

3. User set address and R/W flag in the I2C_DATA_OUT register. Then set I2C_WRITE flag

4. I2C_ACK flag is set by the I2C module depending on the slave response

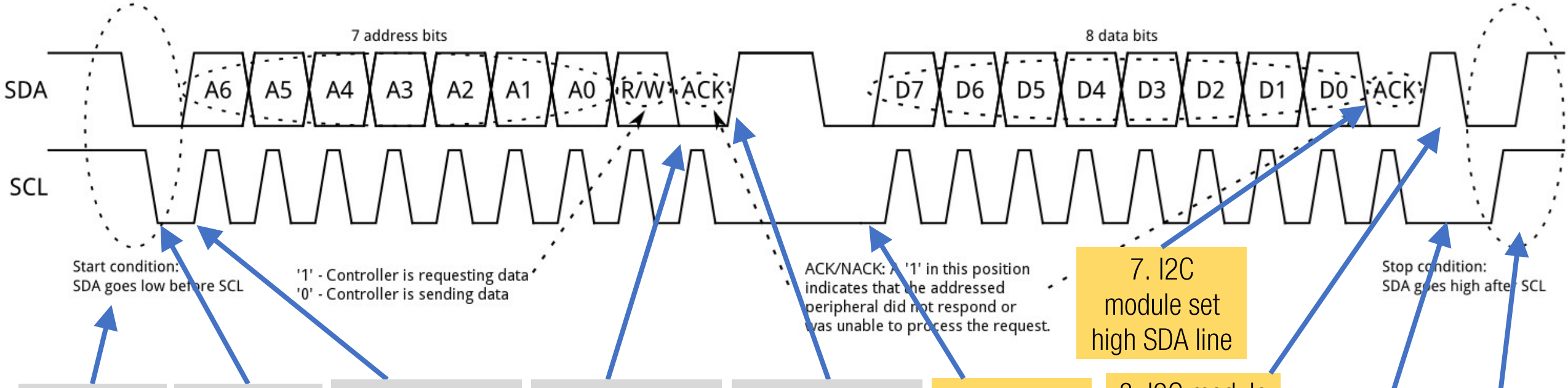
5. I2C_WRITE flag is cleared when transmission is done & interrupt is posted if enabled

6. Repeat steps 3-5 for each byte in the data frame

N-1. User set I2C_STOP flag to finish data frame

N. I2C module clears I2C_STOP flag when done

I2C (Registers & bit protocol during reading)



1. User set I2C_START flag

2. I2C module clears I2C_START flag when done

3. User set address and R/W flag in the I2C_DATA_OUT register. Then set I2C_WRITE flag

4. I2C_ACK flag is set by the I2C module depending on the slave response

5. I2C_WRITE flag is cleared when transmission is done & interrupt is posted if enabled

6. User set the I2C_READ flag to tell the I2C module to read a byte

7. I2C module set high SDA line

8. I2C module clears I2C_READ flag when done & interrupt is posted if enabled. User must read I2C_DATA_IN register

N-1. User set I2C_STOP flag to finish data frame

N. I2C module clears I2C_STOP flag when done

Steps 1-5, N-1, and N are the same as writing

I2C Registers

Register name	Bit size	Description
I2C_BITRATE	32	Clock divider to generate the required transmission rate. SCL_clock = FACTOR * CPU_Clock / I2C_BITRATE
I2C_DATA_OUT	8	This register must be used by the user after the start condition to setup the slave address and the read/write flag (LSB bit). In a writing cycle, the user must write in this register the byte to be transmitted. To transmit the byte written in this register, user must set the I2C_WRITE flag in the I2C_CTRL register.
I2C_DATA_IN	8	In a reading cycle, the user must read this register only when the I2C_READ flag is cleared in the I2C_CTRL register. Reading this register when the I2C_READ flag is high returns an invalid value.
I2C_CTRL	8	I2C configuration and status flags

I2C Registers

I2C_CTRL

7	6	5	4	3	2	1	0
I2C_ON	I2C_START	I2C_STOP	I2C_WRITE	I2C_READ	I2C_ACK	I2C_I_MSK	-

- I2C_ON:
  - 0: I2C disabled
  - 1: I2C ON
- I2C_START: Start condition
  - If this bit is set, a start condition is forced to the SDA and SCL lines, and a I2C transmission begins. This flag is cleared automatically by the I2C module when the start condition is done, and the module is ready to accept the slave address.
- I2C_STOP: Stop condition
  - If this bit is set, a stop condition is forced to the SDA and SCL lines, and the I2C transmission ends. This flag is cleared automatically by the I2C module when the stop condition is done. Then the SDA and SCL lines comes high, and the module goes to the idle state.
- I2C_WRITE: Start a byte writing
  - After the I2C_DATA_OUT register is set by the user, this bit must set to tell the I2C module to write the byte and check the acknowledge. This flag is cleared automatically by the I2C module after the ACK bit is received, in this case the module is ready to write a new byte or to stop the transmission.
- I2C_READ: Start a byte reading.
  - If this bit is set, the I2C module reads one byte and responds with an acknowledge to the slave device. Once the reading is done, the I2C_DATA_IN register holds the most recent data and the I2C_READ flag is cleared automatically by the module.
- I2C_ACK: Acknowledge bit
  - The value of this bit depends on the slave response. If this bit is low, no acknowledge has been received. If this bit is high, the transmission was successful.
- IC2_I_MSK: I2C interrupt mask.
  - When this bit is set, the I2C interrupt is generated any time one byte is read or written. When this bit is cleared, no interrupts are generated.

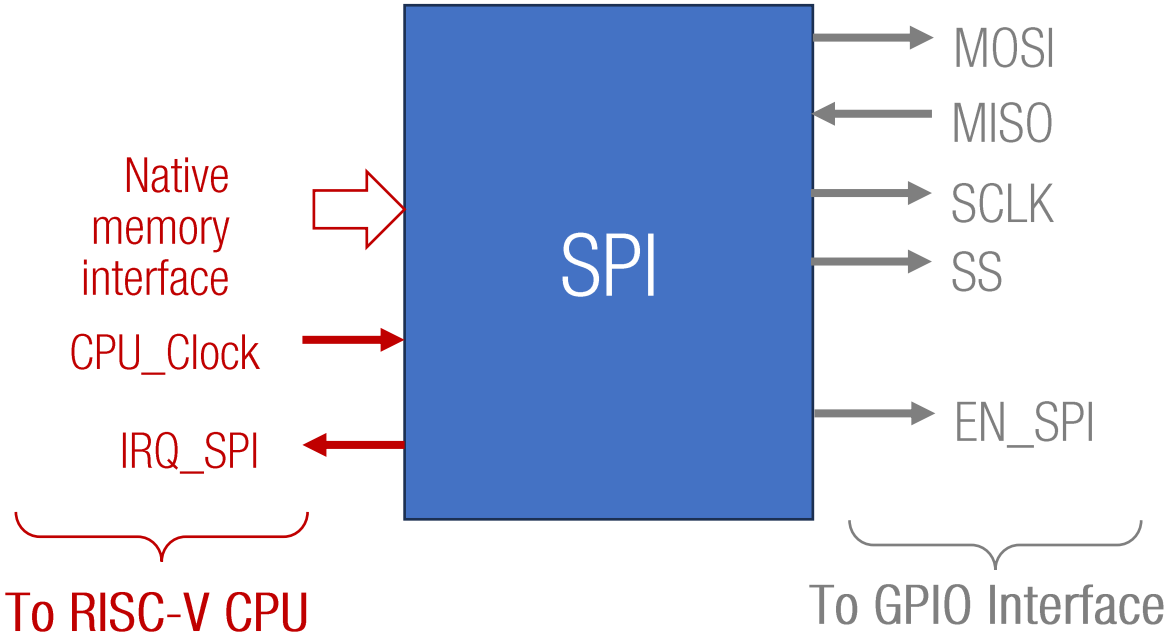
- Important issues:
  - You must establish an equation for SCL_clock = FACTOR * CPU_Clock / I2C_BITRATE depending on the CPU_Clock frequency.
  - You must check the allowed modes for your design, depending on the CPU_Clock frequency

Mode	Highest bit rate	Direction
Standard Mode	0,1 Mbit/s	Bidireccional
Fast Mode	0,4 Mbit/s	Bidireccional
Fast Mode Plus	1,0 Mbit/s	Bidireccional
High Speed Mode	3,4 Mbit/s	Bidireccional
Ultra Fast-mode	5,0 Mbit/s	Unidireccional

SPI Specification

SPI

- Work only as a master device
- Configurable SPI clock frequency
- Fullduplex transmission
- Support: 8-, 16-, 24-, or 32-bits data
- Configurable bit order
- 4 modes of operation
- Interrupts are generated when a data is transmitted

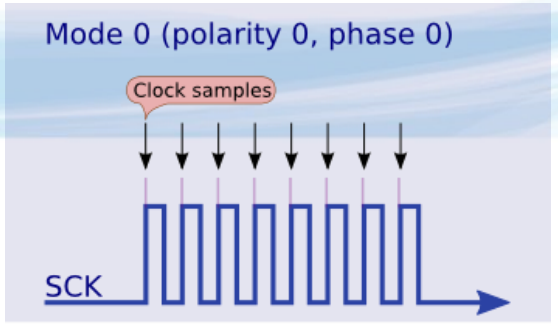


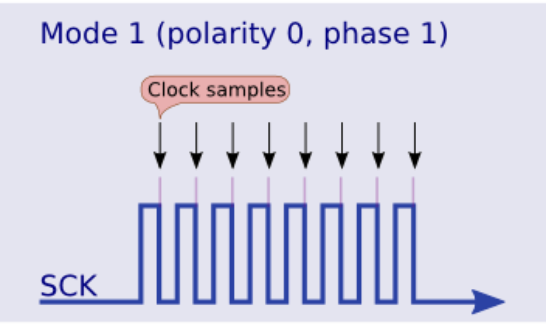
SPI Modes

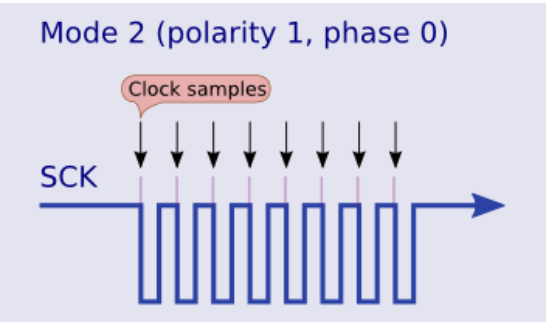
- Data is sampled when:
  - Clock Phase (CPHA)
    - CPHA=1: Sampling on SCLK rising (⬆)
    - CPHA=0: Sampling on SCLK falling (⬇)
  - Clock Polarity (CPOL)
    - CPOL=1: SCLK is high (1) when idle
    - CPOL=0: SCLK is low (0) when idle

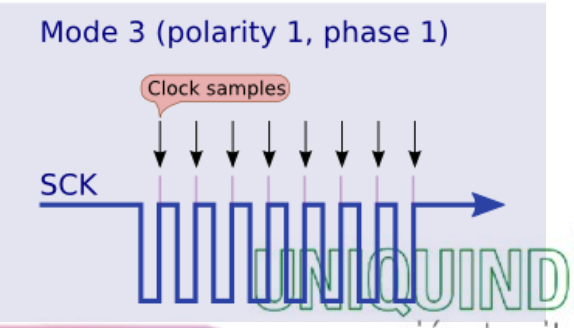
Mode	Clock Polarity (CPOL)	Clock Phase (CPHA)	Output Edge	Data Capture
SPI_MODE0	0	0	Falling	Rising
SPI_MODE1	0	1	Rising	Falling
SPI_MODE2	1	0	Rising	Falling
SPI_MODE3	1	1	Falling	Rising

Tomado de https://diyi0t.com/spi-tutorial-for-arduino-and-esp8266/









SPI (Registers & bit protocol during reading or writing)

1. User sets SPI_FRAME_START flag in SPI_CTRL register to start a frame. Then the SPI module sets the SS line and prepares MOSI and SCLK lines.
2. User write data to SPI_DATA_OUT register. This step can be skipped during a read-only frame.
3. User set the SPI_START flag in the SPI_CTRL register to perform a transmission. This action performs a simultaneous reading and writing cycle providing fullduplex transmission.
4. The SPI module clears SPI_START flag when the transmission is done and generates an interrupt request if enabled.
5. Repeat steps 2 & 3 for all data in the frame.
6. User clears SPI_FRAME_START flag in SPI_CTRL register to stop the frame. This action must return the MOSI, SCLK and SS lines to their idle states according to the selected mode.

SPI Registers

Register name	Bit size	Description
SPI_BITRATE	32	Clock divider to generate the required transmission rate. SPI_clock = FACTOR * CPU_Clock / SPI_BITRATE
SPI_DATA_OUT	32	The user must write in this register the data to be transmitted. It could be 8-, 16-, 24-, or 32-bits value.
SPI_DATA_IN	32	This register holds the last data read from the slave. The user must read this register only when the SPI_START flag is cleared in the SPI_CTRL register. Reading this register when the SPI_START flag is high returns an invalid value.
SPI_CTRL	9	SPI configuration and status flags

Note: SPI module performs a full duplex transmission, i.e., reading and writing is performed simultaneously in the register SPI_DATA_OUT and SPI_DATA_IN. To start a transmission (reading and writing), user must set the SPI_START flag in the SPI_CTRL register and wait until SPI_START flag is cleared. When the SPI_START flag is low, the transmission is done.

SPI Registers

SPI_CTRL

8	7	6	5	4	3	2	1	0
SPI_ON	SPI_MODE		SPI_BIT_ORDER	SPI_DATA_LEN		SPI_FRAME_START	SPI_START	SPI_I_MSK

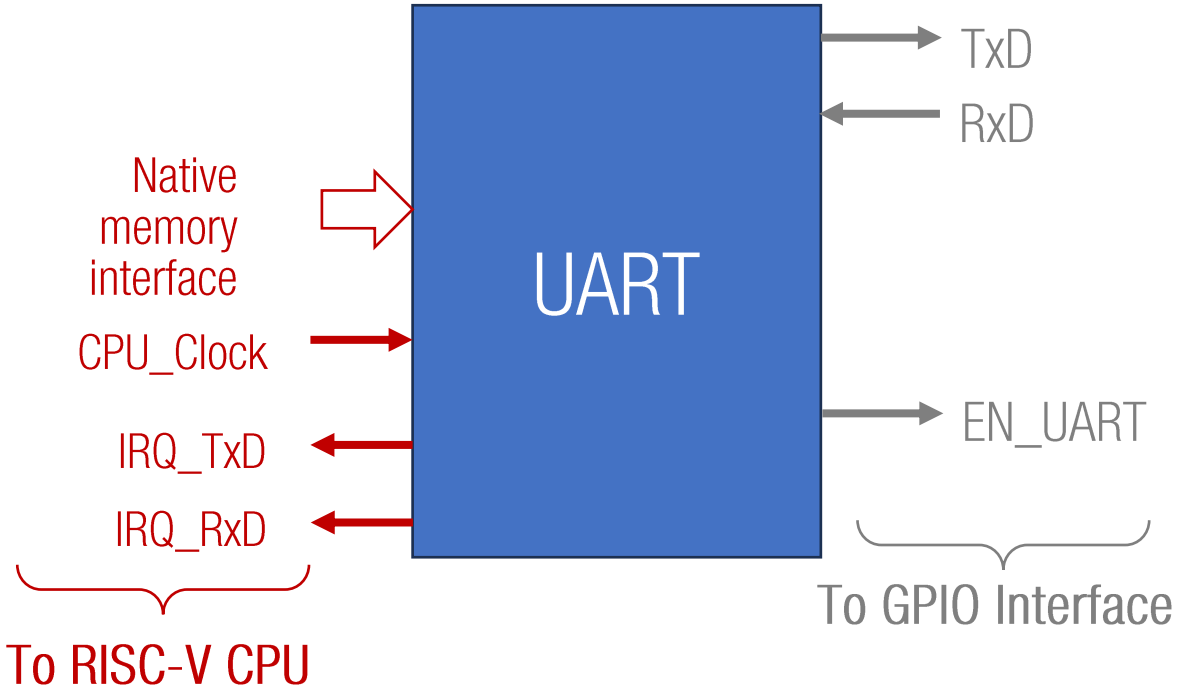
- SPI_ON:
  - 0: SPI disabled
  - 1: SPI ON
- SPI_MODE:
  - 00: Mode 0
  - 01: Mode 1
  - 10: Mode 2
  - 11: Mode 3
- SPI_BIT_ORDER:
  - 0: LSB First
  - 1: MSB First
- SPI_DATA_LEN:
  - 00: 8bit
  - 01: 16bit
  - 10: 24bit
  - 11: 32bit
- SPI_FRAME_START
  - If this bit is set, a frame is started, then the SPI module prepares the MOSI and SCLK to the initial state and set the SS line. When this bit is cleared, the SPI module goes to the idle state.
- SPI_START
  - If this bit is set, the SPI modules starts a transmission writing the data in the SPI_DATA_OUT register to the slave and reading simultaneously the data from slave and modifying the SPI_DATA_IN register. This flag is cleared automatically by the SPI module after a transmission is done.
- SPI_I_MSK: SPI interrupt mask.
  - When this bit is set, the SPI interrupt is generated after a data transmission. When this bit is cleared, no interrupts are generated.

- Important issues:
  - You must establish an equation for SPI_clock = FACTOR * CPU_Clock / SPI_BITRATE depending on the CPU_Clock frequency.
  - You must determine the highest allowed bit rate in your design.

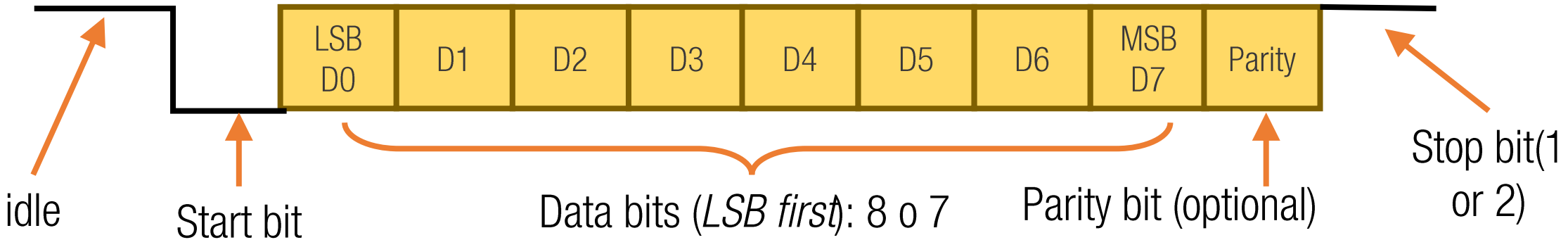
UART Specification

UART

- Configurable UART clock frequency
- Fullduplex transmission
- 8- or 7-bit transmissions
- Configurable parity and stop bits
- Interrupts are generated when a data is transmitted or received



UART (Universal Asynchronous Receiver-Transceiver)



- Supported bit rates:
  - 300, 600, 1200, 1800, 2400, 4800, 9600, 19200, 38400, 57600, 115200

UART Registers

Register name	Bit size	Description
UART_BITRATE	32	Clock divider to generate the required transmission rate. UART_clock = FACTOR * CPU_Clock / UART_BITRATE
UART_DATA_OUT	8	In a writing cycle, the user must write in this register the byte to be transmitted. To transmit the byte written in this register, user must set the UART_WRITE flag in the UART_CTRL register.
UART_DATA_IN	8	This register holds the last byte read from the UART. It is a valid value only when the UART_AVAIL flag is set in the UART_CTRL register. Reading this register when the UART_AVAIL flag is low returns an invalid value.
UART_CTRL	10	UART configuration and status flags

UART Registers

UART_CTRL

9	8	7	6	5	4	3	2	1	0
UART_ON	UART_BITS	UART_PARITY		UART_STOP BITS	UART_ WRITE	UART_ AVAIL	UART_PARITY _ERR	UART_I_MSK	

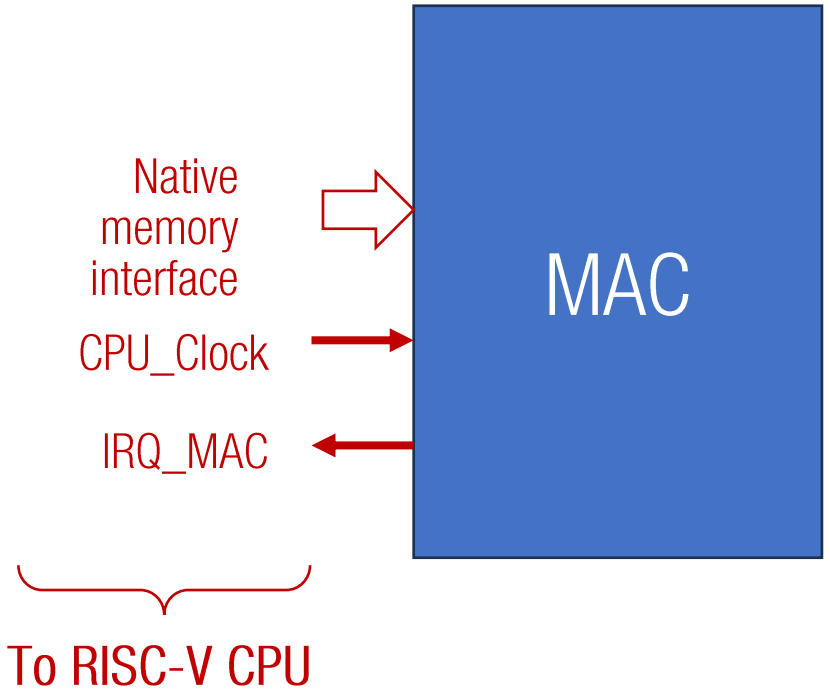
- UART_ON:
  - 0: UART disabled
  - 1: UART ON
- UART_BITS: Number of data bits
  - 0: 7 bits
  - 1: 8 bits
- UART_PARITY: Parity configuration
  - 00: No parity
  - 01: Even parity
  - 10: Odd parity
- UART_STOPBITS: Number of stop bits
  - 0: 1 bit
  - 1: 2 bits
- UART_WRITE: Start a byte writing
  - After the UART_DATA_OUT register is set by the user, this bit must set to tell the UART module to write the byte. This flag is cleared automatically by the UART module after the data is written.
- UART_AVAIL: Data input available.
  - This is a read-only flag. If this bit is set by the UART module, a new byte has been received and it is ready in the UART_DATA_IN register. This flag is cleared automatically when the user reads the UART_DATA_IN register. If this bit is set and the external hardware attempts to send a new byte, the data is lost.
- UART_PARITY_ERR: Parity error
  - If parity is enabled, this bit is set when parity of the reading data fails. If this bit is low, parity checks was successful.
- UART_I_MSK: UART interrupt mask.
  - 00: No interrupts
  - 01: Interrupts only for data reading
  - 10: Interrupts only for data writing
  - 11: Interrupts for both reading and writing.

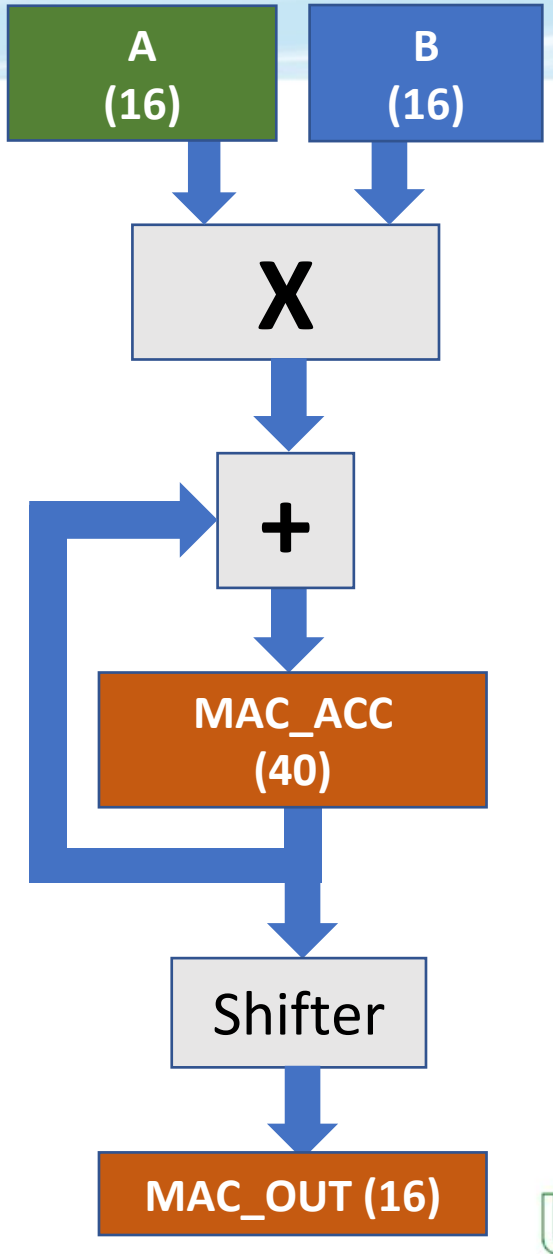
- Important issues:
  - You must establish an equation for UART_clock = FACTOR * CPU_Clock / UART_BITRATE depending on the CPU_Clock frequency for all standardized baud rates

Multiply-Accumulate (MAC) Unit

MAC Unit

- 16-bit input operands
- 16-bit output with automatic fixed-point removal
- 40-bit internal accumulator





MAC Registers

Register name	Bit size	Description
MAC_INA	32	Input A
MAC_INB	32	Input B
MAC_ACCL	32	LSB bits of the accumulator [31:0]
MAC_ACCH	8	MSB bits of the accumulator [39:32]
MAC_OUT	16	[Accumulator << SHIFT] [31:16]
MAC_CTRL	8	MAC configuration and status flags

MAC Registers

MAC_CTRL

7	6	5	4	3	2	1	0
MAC_ON	MAC_SHIFTER			MAC_INPUT_MODE		MAC_START	MAC_I_MSK

- MAC_ON:
  - 0: MAC disabled
  - 1: MAC ON
- MAC_SHIFTER: Number of bits to shift left applied to the accumulator
- MAC_INPUT_MODE:

  Specify how the MAC_INA and MAC_INB registers are feeding into the MAC unity
  - 00: Input A is the lowest 16 bits of MAC_INA and Input B is the highest 16 bits of MAC_INA. MAC_INB register is ignored. Compute a single MAC
  - 01: Input A is the lowest 16 bits of MAC_INA and Input B is the lowest 16 bits of MAC_INB. Compute a single MAC
  - 10: Input A is the highest16 bits of MAC_INA and Input B is the highest 16 bits of MAC_INB. Compute a single MAC
  - 11: Compute two MAC. The first one using the lowest 16 bits of MAC_INA and MAC_INB. The second one using the highest 16bits of MAC_INA and MAC_INB
- MAC_START: Start MAC computation
  - After setting MAC_INA and MAC_INB, the user must set this bit to tell the MAC unit to start computation. This flag is cleared automatically by the MAC module after completion. If MAC_INPUT_MODE=11, this bit is cleared only after two MAC computations are done.
- MAC_I_MSK: MAC interrupt mask.
  - 0: No interrupts
  - 1: Interrupt when MAC is done

UNIVERSIDAD DEL QUINDÍO

ALTA CALIDAD

Res.MEN 014915 - 02 AGO 2022

CERTIFICADO POR SU COMPROMISO RESPONSABILIDAD SOCIAL FENALCO SOLIDARIO